

Course Project: Improving Tetris

Your Goal: From Prototype to High-Quality Software

- The starting Tetris code is intentionally flawed: it contains bugs, global state, unclear names, duplicated logic, and coupled responsibilities.
- Your job is not merely to make it run, but to turn it into a **High-Quality Software Product (HQSP)**:
 - Easy to extend with new features safely
 - Easy to diagnose and fix without breaking existing behavior
 - Protected by automated tests and clear

Why Start With Bad Code?

- Real software is rarely clean when you first encounter it.
- Requirements change after code has already been written.
- Fixing one issue can create another when there is no test or design.
- Software engineers improve existing systems safely instead of rewriting everything blindly.

Our Team Project Journey

Understand the prototype

- Find and reproduce bugs
- Add automated tests
- Refactor without changing behavior
- Introduce software design
- Add features safely
- Deliver a maintainable application

- We will improve the same code throughout the semester.
- Each course topic will give us another tool for improving it.

1. Fix Bugs

- First, learn what the program currently does.
- Run both `tetris_ver1.py` and `tetris_ver2.py`.
- Reproduce each bug before changing the code.
- Write down:
 - Steps that trigger the problem
 - Expected behavior
 - Actual behavior
 - Likely location of the defect

Known Bug Areas: Setup and Execution

- The shell scripts assume tools and environments that may not be configured.
- A script may execute the wrong Python file.
- Package installation may use a different Python from the one running the game.
- A script may work only when launched from one specific directory.

These are starting clues, not a complete bug list.

Known Bug Areas: Game Logic

- Line removal has boundary problems near the top of the board.
- A completed line may not shift every required row or clear the new top row correctly.
- Multiple completed lines and consecutive completed lines need careful verification.
- Collision, rotation, game-over, and scoring behavior may contain additional edge cases.

Do not trust the comments. Confirm behavior by running the code and writing tests.

Bug-Fix Expectations

- Fix the cause, not only the visible symptom.
- Keep each bug fix small and focused.
- Protect each fix with an automated test.
- Do not combine unrelated cleanup with a bug-fix commit.
- Explain why the change is correct.

2. Add Tests

- Tests are the safety net for the rest of the semester.
- Decouple board and game logic from Pygame rendering and keyboard input so `pytest` can run fast without opening a window.
- Keep tests readable as executable specifications of expected behavior.

| Testing Levels to Implement

- **Unit tests:** Verify individual functions, classes, or modules.
- **Integration tests:** Verify that multiple modules work together.
- **Regression tests:** Preserve behavior that was previously fixed or working.
- **Acceptance tests:** Confirm that the implemented system meets its requirements.

| What Should We Test?

- Board initialization and dimensions
- Collision with walls, the floor, and existing blocks
- Movement and rejected movement
- Rotation and invalid rotation
- Freezing a piece into the board
- Clearing zero, one, and multiple lines
- Score calculation and invalid inputs
- Game-over conditions

| Test More Than the Happy Path

- Test boundaries: first row, last row, left wall, and right wall.
- Test empty, full, and nearly full boards.
- Test consecutive line clears.
- Test invalid inputs and impossible state transitions.
- Control randomness with a seed, dependency injection, or a test double.
 - For example, pass a fixed sequence of tetrominoes into the game instead of selecting random pieces during a test.

A Good Regression-Test Workflow

Observe a failure

- Write a test that reproduces it
- Confirm the test fails for the expected reason
- Make the smallest correct fix
- Confirm all tests pass
- Refactor only after the behavior is protected

| Testing Expectations

- Tests must verify behavior, not implementation details.
- A passing test suite is required before merging changes.
- Tests should be deterministic and independent.
- Test names should describe the scenario and expected result.
- Coverage is useful evidence, but a high percentage does not guarantee good tests.

3. Add Software Design

- Next week, we begin introducing design to the prototype.
- Design is not adding classes everywhere.
- Design assigns clear responsibilities and makes expected changes safer.
- We will use evidence from bugs, tests, and feature requests to guide design decisions.

Design Problems to Investigate

- Global variables expose and couple the entire game state.
- Game rules, input handling, drawing, timing, and persistence are mixed together.
- Functions depend on hidden state instead of explicit inputs and outputs.
- Tetromino data, board logic, scoring, and display have unclear ownership.
- Adding a new scoring rule or feature may require editing unrelated code.

Possible Design Directions

- Encapsulate board, piece, and game state.
- Separate domain logic from the Pygame user interface.
- Give functions and classes one clear responsibility.
- Replace magic numbers and unclear names with domain concepts.
- Introduce patterns only when they solve a demonstrated change problem.
- Record significant choices and trade-offs in an ADR.

These are design questions, not a required final class diagram.

Refactoring Expectations

- Preserve observable behavior unless a requirement or bug fix changes it.
- Refactor in small steps while tests remain green.
- Avoid a complete rewrite: evolve the existing system and preserve its history.
- Remove code smells for a reason, not merely to satisfy a style rule.
- Be able to explain how each design change improves maintainability or testability.

Adding Features Throughout the Semester

- New requirements will test whether the design is flexible.
- Possible changes include:
 - Alternative scoring strategies
 - New game modes or difficulty levels
 - Saved high scores
 - Improved controls and display
 - Additional piece or board behavior
- A good design makes these changes local, understandable, and testable.

How to Work With AI

- Use AI to investigate code, generate alternatives, and draft tests or documentation.
- Give AI the relevant requirements, boundaries, existing design, and test expectations.
- Use AI to help automate repeatable tests rather than asking it to inspect the program informally each time.
- Never accept generated code without reading, running, and testing it.
- Do not ask AI to rewrite the whole project and submit the result as your work.

Team and Git Expectations

- Work on a short-lived branch for one bug, test group, refactoring, or feature.
- Make small commits with messages that explain the change.
- Use pull requests for review and discussion.
- Pull before starting work and communicate before resolving shared conflicts.
- Do not commit virtual environments, caches, generated files, credentials, or secrets.

Semester Evidence Expected From Your Team

- A maintained issue log showing the problems discovered and resolved
- A growing automated test suite and summary of the behavior it protects
- Git history showing incremental contributions from the team
- A collection of significant refactoring explanations and ADRs
- Current user and architecture documentation

Definition of Done for a Change

- Apply this checklist to each change or pull request.
- The requirement and acceptance criteria are clear.
- The implementation is understandable and appropriately scoped.
- Relevant automated tests pass.
- Existing behavior has not regressed.
- Documentation and design records are updated when needed.

What We Do Not Expect

- Perfect code in the first week
- A complete redesign before understanding the prototype
- Classes or design patterns added without a concrete reason
- Large AI-generated changes that the team cannot explain
- Features added while known regressions are ignored
- One team member doing all implementation and design work

The Final Expectation

- By the end of the semester, the application should be more correct, tested, understandable, and adaptable than the starting prototype.
- More importantly, your team should be able to explain:
 - Which problems you discovered
 - How tests protected the work
 - Why the design evolved
 - What trade-offs you accepted
 - How you know the final system works

We Will Build Quality One Change at a Time

The goal is not to hide the bad starting code. The goal is to demonstrate professional software engineering by improving it throughout the semester.